1.3 CRITÉRIOS DE AVALIAÇÃO DE LINGUAGENS

Conforme mencionado, o objetivo deste livro é examinar os conceitos fundamentais das diversas construções e capacidades das linguagens de programação. Iremos também avaliar esses recursos, focando em seu impacto no processo de desenvolvimento de software, incluindo a manutenção. Para isso, precisamos de um conjunto de critérios de avaliação. Tal lista é necessariamente controversa, porque é praticamente impossível fazer dois cientistas da computação concordarem com o valor de certas características das linguagens em relação às outras. Apesar dessas diferenças, a maioria concordaria que os critérios discutidos nas subseções a seguir são importantes.

Algumas das características que influenciam três dos quatro critérios mais importantes são mostradas na Tabela 1.1, e os critérios propriamente ditos são discutidos nas seções seguintes[1]. Note que apenas as características mais importantes são incluídas na tabela, espelhando a discussão nas subseções seguintes. Alguém poderia argumentar que, se fossem consideradas características menos importantes, praticamente todas as posições da tabela poderiam ser marcadas.

Note que alguns dos critérios são amplos – e, de certa forma, vagos – como a facilidade de escrita, enquanto outros são construções específicas de linguagens, como o tratamento de exceções. Apesar de a discussão parecer implicar que os critérios têm uma importância idêntica, não é o caso.

Tabela 1.1 Critérios de avaliação de linguagens e as características que os afetam

	CRITÉRIOS		
Característica	**LEGIBILIDADE**	**FACILIDADE DE ESCRITA**	**CONFIABILIDADE**
Simplicidade	•	•	•
Ortogonalidade	•	•	•
Tipos de dados	•	•	•
Projeto de sintaxe	•	•	•
Suporte para abstração		•	•
Expressividade		•	•
Verificação de tipos			•
Tratamento de exceções			•
Apelidos restritos			•

[1] O quarto critério principal é o custo, que não foi incluído na tabela porque é apenas superficialmente relacionado aos outros três critérios e às características que os influenciam.

1.3.1 Legibilidade

Um dos critérios mais importantes para julgar uma linguagem de programação é a facilidade com a qual os programas podem ser lidos e entendidos. Antes de 1970, o desenvolvimento de software era amplamente pensado em termos da escrita de código. As principais características positivas das linguagens de programação eram a eficiência e a legibilidade de máquina. As construções das linguagens foram projetadas mais do ponto de vista do computador do que do dos usuários. Nos anos 1970, entretanto, o conceito de ciclo de vida de software (Booch, 1987) foi desenvolvido; a codificação foi relegada a um papel muito menor, e a manutenção foi reconhecida como uma parte principal do ciclo, particularmente em termos de custo. Como a facilidade de manutenção é determinada, em grande parte, pela legibilidade dos programas, a legibilidade se tornou uma medida importante da qualidade dos programas e das linguagens de programação, o que representou um marco na sua evolução. Ocorreu uma transição de foco bem definida: da orientação à máquina à orientação às pessoas

A legibilidade deve ser considerada no contexto do domínio do problema. Por exemplo, se um programa que descreve uma computação é escrito em uma linguagem que não foi projetada para tal uso, ele pode não ser natural e desnecessariamente complexo, tornando complicada sua leitura (quando em geral seria algo simples).

As subseções a seguir descrevem características que contribuem para a legibilidade de uma linguagem de programação.

1.3.1.1 Simplicidade geral

A simplicidade geral de uma linguagem de programação afeta fortemente sua legibilidade. Uma linguagem com muitas construções básicas é mais difícil de aprender do que uma com poucas. Os programadores que precisam usar uma linguagem grande aprendem um subconjunto dessa linguagem e ignoram outros recursos. Esse padrão de aprendizagem é usado como desculpa para a grande quantidade de construções de uma linguagem, mas o argumento não é válido. Problemas de legibilidade ocorrem sempre que o autor de um programa aprender um subconjunto diferente daquele com o qual o leitor está familiarizado.

Outra característica de uma linguagem de programação que pode ser um complicador é a **multiplicidade de recursos** – ou seja, haver mais de uma maneira de realizar uma operação. Por exemplo, em Java um usuário pode incrementar uma simples variável inteira de quatro maneiras:

```
count = count + 1
count += 1
count++
++count
```

Apesar de as últimas duas sentenças terem significados um pouco diferentes uma da outra e em relação às duas primeiras em alguns contextos, as quatro

têm o mesmo significado quando usadas em expressões isoladas. Essas variações são discutidas no Capítulo 7.

Um terceiro problema em potencial é a **sobrecarga de operadores**, na qual um operador tem mais de um significado. Apesar de ser útil, pode levar a uma redução da legibilidade se for permitido aos usuários criar suas próprias sobrecargas e eles não o fizerem de maneira sensata. Por exemplo, é aceitável sobrecarregar o operador + para usá-lo tanto para a adição de inteiros quanto para a adição de valores de ponto flutuante. Na verdade, essa sobrecarga simplifica uma linguagem ao reduzir o número de operadores. Entretanto, suponha que o programador definiu o símbolo + usado entre vetores de uma única dimensão para significar a soma de todos os elementos de ambos os vetores. Como o significado usual da adição de vetores é bastante diferente, isso tornaria o programa mais confuso, tanto para o autor quanto para os leitores do programa. Um exemplo ainda mais extremo da confusão em programas seria um usuário definir que o + entre dois operandos do tipo vetor seja a diferença entre os primeiros elementos de cada vetor. A sobrecarga de operadores é discutida com mais detalhes no Capítulo 7.

A simplicidade em linguagens pode, é claro, ser levada muito ao extremo. Por exemplo, a forma e o significado da maioria das sentenças de uma linguagem *assembly* são modelos de simplicidade, como você pode ver quando considera as sentenças que aparecem na próxima seção. Essa simplicidade extrema, entretanto, torna menos legíveis os programas escritos em linguagem *assembly*.

Devido à falta de sentenças de controle mais complexas, a estrutura de um programa é menos óbvia; e como as sentenças são simples, são necessárias mais do que em programas equivalentes escritos em uma linguagem de alto nível. Os mesmos argumentos se aplicam para os casos menos extremos de linguagens de alto nível com construções inadequadas de controle e de estruturas de dados.

1.3.1.2 Ortogonalidade

Ortogonalidade em uma linguagem de programação significa que um conjunto relativamente pequeno de construções primitivas pode ser combinado a um número relativamente pequeno de formas para construir as estruturas de controle e de dados da linguagem. Além disso, cada possível combinação de primitivas é legal e significativa. Por exemplo, considere os tipos de dados. Suponha que uma linguagem tenha quatro tipos primitivos de dados (inteiro, ponto flutuante, ponto flutuante de dupla precisão e caractere) e dois operadores de tipo (vetor e ponteiro). Se os dois operadores de tipo puderem ser aplicados a eles mesmos e aos quatro tipos de dados primitivos, um grande número de estruturas de dados pode ser definido.

O significado de um recurso de linguagem ortogonal é independente do contexto de sua aparição em um programa (o nome ortogonal vem do conceito matemático de vetores ortogonais, independentes uns dos outros). A ortogonalidade vem de uma simetria de relacionamentos entre primitivas. Uma falta de ortogonalidade leva a exceções às regras de linguagem. Por exemplo, deve ser possível, em uma linguagem de programação que possibilita o uso de

ponteiros, definir que um aponte para qualquer tipo específico definido na linguagem. Entretanto, se não for permitido aos ponteiros apontar para vetores, muitas estruturas de dados potencialmente úteis definidas pelos usuários não poderiam ser definidas.

Podemos ilustrar o uso da ortogonalidade como um conceito de projeto ao comparar um aspecto das linguagens *assembly* dos mainframes da IBM com a série VAX de minicomputadores. Consideramos uma situação simples: adicionar dois valores inteiros de 32-bits que residem na memória ou nos registradores e substituir um dos valores pela soma. Os mainframes IBM têm duas instruções para esse propósito, com a forma

```
A Reg1, memory_cell
AR Reg1, Reg2
```

onde `Reg1` e `Reg2` representam registradores. A semântica desses é

```
Reg1 ← contents(Reg1) + contents(memory_cell)
Reg1 ← contents(Reg1) + contents(Reg2)
```

A instrução de adição VAX para valores inteiros de 32-bits é

```
ADDL operand_1, operand_2
```

cuja semântica é

```
operand_2 ← contents(operand_1) + contents(operand_2)
```

Nesse caso, cada operando pode ser registrador ou uma célula de memória.

O projeto de instruções VAX é ortogonal no sentido de que uma instrução pode usar tanto registradores quanto células de memória como operandos. Existem duas maneiras para especificar operandos, as quais podem ser combinadas de todas as maneiras possíveis. O projeto da IBM não é ortogonal. Apenas duas combinações de operandos são legais (dentre quatro possibilidades), e ambas necessitam de instruções diferentes, `A` e `AR`. O projeto da IBM é mais restrito e tem menor facilidade de escrita. Por exemplo, você não pode adicionar dois valores e armazenar a soma em um local de memória. Além disso, o projeto da IBM é mais difícil de aprender por causa das restrições e da instrução adicional.

A ortogonalidade é fortemente relacionada à simplicidade: quanto mais ortogonal o projeto de uma linguagem, menor é o número necessário de exceções às regras da linguagem. Menos exceções significam um maior grau de regularidade no projeto, o que torna a linguagem mais fácil de aprender, ler e entender. Qualquer um que tenha aprendido uma parte significativa da língua inglesa pode testemunhar sobre a dificuldade de aprender suas muitas exceções de regras (por exemplo, *i* antes de *e* exceto após *c*).

Como exemplos da falta de ortogonalidade em uma linguagem de alto nível, considere as seguintes regras e exceções em C. Apesar de C ter duas formas de tipos de dados estruturados, vetores e registros (**`structs`**), os registros podem ser retornados por funções, mas os vetores não. Um membro de uma

estrutura pode ser de qualquer tipo de dados, exceto **void** ou uma estrutura do mesmo tipo. Um elemento de um vetor pode ser qualquer tipo de dados, exceto **void** ou uma função. Parâmetros são passados por valor, a menos que sejam vetores, o que faz com que sejam passados por referência (porque a ocorrência de um nome de um vetor sem um índice em um programa em C é interpretada como o endereço do primeiro elemento desse vetor).

Como um exemplo da dependência do contexto, considere a seguinte expressão em C

```
a + b
```

Ela significa que os valores de `a` e `b` são obtidos e adicionados juntos. Entretanto, se `a` for um ponteiro, afeta o valor de `b`. Por exemplo, se `a` aponta para um valor de ponto flutuante que ocupa quatro bytes, o valor de `b` deve ser ampliado – nesse caso, multiplicado por 4 – antes que seja adicionado a `a`. Logo, o tipo de `a` afeta o tratamento do valor de `b`. O contexto de `b` afeta seu significado.

Muita ortogonalidade também pode causar problemas. Talvez a linguagem de programação mais ortogonal seja o ALGOL 68 (van Wijngaarden et al.,1969). Cada construção de linguagem no ALGOL 68 tem um tipo, e não existem restrições nesses tipos. Além disso, mais construções produzem valores. Essa liberdade de combinações permite construções extremamente complexas. Por exemplo, uma sentença condicional pode aparecer no lado esquerdo de uma atribuição, com declarações e outras sentenças diversas, desde que o resultado seja um endereço. Essa forma extrema de ortogonalidade leva a uma complexidade desnecessária. Como as linguagens necessitam de um grande número de tipos primitivos, um alto grau de ortogonalidade resulta em uma explosão de combinações. Então, mesmo se as combinações forem simples, seu número já leva à complexidade.

Simplicidade em uma linguagem é, ao menos em parte, o resultado de uma combinação de um número relativamente pequeno de construções primitivas e um uso limitado do conceito de ortogonalidade.

Algumas pessoas acreditam que as linguagens funcionais oferecem uma boa combinação de simplicidade e ortogonalidade. Uma linguagem funcional, como LISP, é uma na qual as computações são feitas basicamente pela aplicação de funções para parâmetros informados. Em contraste a isso, em linguagens imperativas como C, C++ e Java, as computações são normalmente especificadas com variáveis e sentenças de atribuição. As linguagens funcionais oferecem a maior simplicidade de um modo geral, porque podem realizar tudo com uma construção, a chamada à função, a qual pode ser combinada com outras funções de maneiras simples. Essa elegância simples é a razão pela qual alguns pesquisadores de linguagens são atraídos pelas linguagens funcionais como principal alternativa às complexas linguagens não funcionais como C++. Outros fatores, como a eficiência, entretanto, têm prevenido que as linguagens funcionais sejam mais utilizadas.

1.3.1.3 Tipos de dados

A presença de mecanismos adequados para definir tipos e estruturas de dados é outro auxílio significativo à legibilidade. Por exemplo, suponha que um tipo numérico seja usado como uma *flag* porque não existe nenhum tipo booleano na linguagem. Em tal linguagem, poderíamos ter uma atribuição como:

```
timeOut = 1
```

O significado dessa sentença não é claro. Em uma linguagem que inclui tipos booleanos, teríamos:

```
timeOut = true
```

O significado dessa sentença é perfeitamente claro.

1.3.1.4 Projeto da sintaxe

A sintaxe, ou forma, dos elementos de uma linguagem tem um efeito significativo na legibilidade dos programas. A seguir, estão dois exemplos de escolhas de projeto sintáticas que afetam a legibilidade:

- *Formato dos identificadores*. Restringir os identificadores a tamanhos muito curtos piora a legibilidade. Se os identificadores podem ter no máximo seis caracteres, como no Fortran 77, é praticamente impossível usar nomes conotativos para variáveis. Um exemplo mais extremo é o ANSI* BASIC (ANSI, 1978b), no qual um identificador poderia ser formado por apenas uma letra ou por uma letra seguida de um dígito. Outras questões de projeto relacionadas ao formato dos identificadores são discutidas no Capítulo 5.
- *Palavras especiais*. A aparência de um programa e sua legibilidade são fortemente influenciadas pela forma das palavras especiais de uma linguagem (por exemplo, **`while`**, **`class`** e **`for`**). O método para formar sentenças compostas, ou grupos de sentenças, é especialmente importante, principalmente em construções de controle. Algumas linguagens têm usado pares casados de palavras especiais ou de símbolos para formar grupos. C e seus descendentes usam chaves para especificar sentenças compostas. Todas essas linguagens sofrem porque os grupos de sentenças são sempre terminados da mesma forma, o que torna difícil determinar qual grupo está sendo finalizado quando um **`end`** ou um } aparece. O Fortran 95 e Ada tornaram isso claro ao usar uma sintaxe distinta para cada tipo de grupo de sentenças. Por exemplo, Ada utiliza **`end if`** para terminar uma construção de seleção e **`end loop`** para terminar uma construção de repetição. Esse é um exemplo do conflito entre a simpli-

* N. de T.: American National Standards Institute, Instituto Nacional Norte–Americano de Padrões.

cidade resultante ao serem usadas menos palavras reservadas, como em C++, e a maior legibilidade que pode resultar do uso de mais palavras reservadas, como em Ada.

Outra questão importante é se as palavras especiais de uma linguagem podem ser usadas como nomes de variáveis de programas. Se puderem, os programas resultantes podem ser bastante confusos. Por exemplo, no Fortran 95, palavras especiais como `Do` e `End` são nomes válidos de variáveis, logo a ocorrência dessas palavras em um programa pode ou não conotar algo especial.

- *Forma e significado*. Projetar sentenças de maneira que sua aparência ao menos indique parcialmente seu propósito é uma ajuda óbvia à legibilidade. A semântica, ou significado, deve advir diretamente da sintaxe, ou da forma. Em alguns casos, esse princípio é violado por duas construções de uma mesma linguagem idênticas ou similares na aparência, mas com significados diferentes, dependendo talvez do contexto. Em C, por exemplo, o significado da palavra reservada **`static`** depende do contexto no qual ela aparece. Se for usada na definição de uma variável dentro de uma função, significa que a variável é criada em tempo de compilação. Se for usada na definição de uma variável fora de todas as funções, significa que a variável é visível apenas no arquivo no qual sua definição aparece; ou seja, não é exportada desse arquivo.

Uma das principais reclamações sobre os comandos de *shell* do UNIX (Raymond, 2004) é que a sua aparência nem sempre sugere sua funcionalidade. Por exemplo, o significado do comando UNIX `grep` pode ser decifrado apenas com conhecimento prévio, ou talvez com certa esperteza e familiaridade com o editor UNIX `ed`. A aparência do `grep` não tem conotação alguma para iniciantes no UNIX. (No `ed`, o comando */regular_expression/* busca uma subcadeia que casa com a expressão regular. Preceder isso com `g` faz ele ser um comando global, especificando que o escopo da busca é o arquivo inteiro que está sendo editado. Seguir o comando com `p` especifica que as linhas contendo a subcadeia casada devem ser impressas. Logo `g/`*regular_expression*`/p`, que pode ser abreviado como `grep`, imprime todas as linhas em um arquivo que contenham subcadeias que casem com a expressão regular.)

1.3.2 Facilidade de escrita

A facilidade de escrita é a medida do quão facilmente uma linguagem pode ser usada para criar programas para um domínio. A maioria das características de linguagem que afetam a legibilidade também afeta a facilidade de escrita. Isso é derivado do fato de que o processo de escrita de um programa requer que o programador releia sua parte já escrita.

Como ocorre com a legibilidade, a facilidade de escrita deve ser considerada no contexto do domínio de problema alvo de uma linguagem. Não é razoável comparar a facilidade de escrita de duas linguagens no contexto de uma aplicação em particular quando uma delas foi projetada para tal aplicação

e a outra não. Por exemplo, as facilidades de escrita do Visual BASIC (VB) e do C são drasticamente diferentes para criar um programa com uma interface gráfica com o usuário, para o qual o VB é ideal. Suas facilidades de escrita também são bastante diferentes para a escrita de programas de sistema, como um sistema operacional, para os quais a linguagem C foi projetada.

As seções seguintes descrevem as características mais importantes que influenciam a facilidade de escrita de uma linguagem.

1.3.2.1 Simplicidade e ortogonalidade

Se uma linguagem tem um grande número de construções, alguns programadores não estarão familiarizados com todas. Essa situação pode levar ao uso incorreto de alguns recursos e a uma utilização escassa de outros que podem ser mais elegantes ou mais eficientes (ou ambos) do que os usados. Pode até mesmo ser possível, conforme destacado por Hoare (1973), usar recursos desconhecidos acidentalmente, com resultados inesperados. Logo, um número menor de construções primitivas e um conjunto de regras consistente para combiná-las (isso é, ortogonalidade) é muito melhor do que diversas construções primitivas. Um programador pode projetar uma solução para um problema complexo após aprender apenas um conjunto simples de construções primitivas.

Por outro lado, muita ortogonalidade pode prejudicar a facilidade de escrita. Erros em programas podem passar despercebidos quando praticamente quaisquer combinações de primitivas são legais. Isso pode levar a certos absurdos no código que não podem ser descobertos pelo compilador.

1.3.2.2 Suporte à abstração

Brevemente, **abstração** significa a habilidade de definir e usar estruturas ou operações complicadas de forma a permitir que muitos dos detalhes sejam ignorados. A abstração é um conceito fundamental no projeto atual de linguagens de programação. O grau de abstração permitido por uma linguagem de programação e a naturalidade de sua expressão são importantes para sua facilidade de escrita. As linguagens de programação podem oferecer suporte a duas categorias de abstrações: processos e dados.

Um exemplo simples da abstração de processos é o uso de um subprograma para implementar um algoritmo de ordenação necessário diversas vezes em um programa. Sem o subprograma, o código de ordenação teria de ser replicado em todos os lugares onde fosse preciso, o que tornaria o programa muito mais longo e tedioso de ser escrito. Talvez o mais importante, se o subprograma não fosse usado, o código que usava o subprograma de ordenação estaria mesclado com os detalhes do algoritmo de ordenação, obscurecendo o fluxo e a intenção geral do código.

Como um exemplo de abstração de dados, considere uma árvore binária que armazena dados inteiros em seus nós. Tal árvore poderia ser implementada em uma linguagem que não oferece suporte a ponteiros e gerenciamento de memória dinâmica usando um monte (*heap*), como no Fortran 77, com o

uso de três vetores inteiros paralelos, onde dois dos inteiros são usados como índices para especificar nós filhos. Em C++ e Java, essas árvores podem ser implementadas utilizando uma abstração de um nó de árvore na forma de uma simples classe com dois ponteiros (ou referências) e um inteiro. A naturalidade da última representação torna muito mais fácil escrever um programa que usa árvores binárias nessas linguagens do que um em Fortran 77. É uma simples questão do domínio da solução do problema da linguagem ser mais próxima do domínio do problema.

O suporte geral para abstrações é um fator importante na facilidade de escrita de uma linguagem.

1.3.2.3 Expressividade

A expressividade em uma linguagem pode se referir a diversas características. Em uma linguagem como APL (Gilman e Rose, 1976), expressividade significa a existência de operadores muito poderosos que permitem muitas computações com um programa muito pequeno. Em geral, uma linguagem expressiva especifica computações de uma forma conveniente, em vez de deselegante. Por exemplo, em C, a notação `count++` é mais conveniente e menor do que `count = count + 1`. Além disso, o operador booleano **`and then`** em Ada é uma maneira conveniente de especificar avaliação em curto-circuito de uma expressão booleana. A inclusão da sentença **`for`** em Java torna a escrita de laços de contagem mais fácil do que com o uso do **`while`**, também possível. Todas essas construções aumentam a facilidade de escrita de uma linguagem.

1.3.3 Confiabilidade

Um programa é dito confiável quando está de acordo com suas especificações em todas as condições. As seguintes subseções descrevem diversos recursos de linguagens que têm um efeito significativo na confiabilidade dos programas em uma linguagem.

1.3.3.1 Verificação de tipos

A **verificação de tipos** é a execução de testes para detectar erros de tipos em um programa, tanto por parte do compilador quanto durante a execução de um programa. A verificação de tipos é um fator importante na confiabilidade de uma linguagem. Como a verificação de tipos em tempo de execução é cara, a verificação em tempo de compilação é mais desejável. Além disso, quanto mais cedo os erros nos programas forem detectados, menos caro é fazer todos os reparos necessários. O projeto de Java requer verificações dos tipos de praticamente todas as variáveis e expressões em tempo de compilação. Isso praticamente elimina erros de tipos em tempo de execução em programas Java. Tipos e verificação de tipos são assuntos discutidos em profundidade no Capítulo 6.

Um exemplo de como a falha em verificar tipos, tanto em tempo de compilação quanto em tempo de execução, tem levado a incontáveis erros de

programa é o uso de parâmetros de subprogramas na linguagem C original (Kernighan e Ritchie, 1978). Nessa linguagem, o tipo de um parâmetro real em uma chamada à função não era verificado para determinar se seu tipo casava com aquele do parâmetro formal correspondente na função. Uma variável do tipo `int` poderia ser usada como um parâmetro real em uma chamada à uma função que esperava um tipo `float` como seu parâmetro formal, e nem o compilador nem o sistema de tempo de execução teriam detectado a inconsistência. Por exemplo, como a sequência de bits que representa o inteiro 23 é essencialmente não relacionada com a sequência de bits que representa o ponto flutuante 23, se um inteiro 23 fosse enviado a uma função que espera um parâmetro de ponto flutuante, quaisquer usos desse parâmetro na função produziriam resultados sem sentido. Além disso, tais problemas são difíceis de serem diagnosticados[2]. A versão atual de C eliminou esse problema ao requerer que todos os parâmetros sejam verificados em relação aos seus tipos. Subprogramas e técnicas de passagem de parâmetros são discutidos no Capítulo 9.

1.3.3.2 Tratamento de exceções

A habilidade de um programa de interceptar erros em tempo de execução (além de outras condições não usuais detectáveis pelo programa), tomar medidas corretivas e então continuar é uma ajuda óbvia para a confiabilidade. Tal facilidade é chamada de **tratamento de exceções**. Ada, C++ e Java incluem diversas capacidades para tratamento de exceções, mas tais facilidades são praticamente inexistentes em muitas linguagens amplamente usadas, como C e Fortran. O tratamento de exceções é discutido no Capítulo 14.

1.3.3.3 Utilização de apelidos

Em uma definição bastante informal, **apelidos** são permitidos quando é possível ter um ou mais nomes para acessar a mesma célula de memória. Atualmente, é amplamente aceito que o uso de apelidos é um recurso perigoso em uma linguagem de programação. A maioria das linguagens permite algum tipo de apelido – por exemplo, dois ponteiros configurados para apontarem para a mesma variável, o que é possível na maioria das linguagens. O programador deve sempre lembrar que trocar o valor apontado por um dos dois ponteiros modifica o valor referenciado pelo outro. Alguns tipos de uso de apelidos, conforme descrito nos Capítulos 5 e 9, podem ser proibidos pelo projeto de uma linguagem.

Em algumas linguagens, apelidos são usados para resolver deficiências nos recursos de abstração de dados. Outras restringem o uso de apelidos para aumentar sua confiabilidade.

---

[2] Em resposta a isso e a outros problemas similares, os sistemas UNIX incluem um programa utilitário chamado `lint`, que verifica os programas em C para checar tais problemas.

1.3.3.4 Legibilidade e facilidade de escrita

Tanto a legibilidade quanto a facilidade de escrita influenciam a confiabilidade. Um programa escrito em uma linguagem que não oferece maneiras naturais para expressar os algoritmos requeridos irá necessariamente usar abordagens não naturais, menos prováveis de serem corretas em todas as situações possíveis. Quanto mais fácil é escrever um programa, mais provavelmente ele estará correto.

A legibilidade afeta a confiabilidade tanto nas fases de escrita quanto nas de manutenção do ciclo de vida. Programas difíceis de ler são também difíceis de escrever e modificar.

1.3.4 Custo

O custo total definitivo de uma linguagem de programação é uma função de muitas de suas características.

Primeiro, existe o custo de treinar programadores para usar a linguagem, que é uma função da simplicidade, da ortogonalidade da linguagem e da experiência dos programadores. Apesar de linguagens mais poderosas não necessariamente serem mais difíceis de aprender, normalmente elas o são.

Segundo, o custo de escrever programas na linguagem. Essa é uma função da facilidade de escrita da linguagem, a qual depende da proximidade com o propósito da aplicação em particular. Os esforços originais de projetar e implementar linguagens de alto nível foram dirigidos pelo desejo de diminuir os custos da criação de software.

Tanto o custo de treinar programadores quanto o custo de escrever programas em uma linguagem podem ser reduzidos significativamente em um bom ambiente de programação. Ambientes de programação são discutidos na Seção 1.8.

Terceiro, o custo de compilar programas na linguagem. Um grande impeditivo para os primeiros usos de Ada era o custo proibitivamente alto da execução dos compiladores da primeira geração. Esse problema foi reduzido com a aparição de compiladores Ada melhores.

Quarto, o custo de executar programas escritos em uma linguagem é amplamente influenciado pelo projeto dela. Uma linguagem que requer muitas verificações de tipos em tempo de execução proibirá uma execução rápida de código, independentemente da qualidade do compilador. Apesar de eficiência de execução ser a principal preocupação no projeto das primeiras linguagens, atualmente é considerada menos importante.

Uma escolha simples pode ser feita entre o custo de compilação e a velocidade de execução do código compilado. **Otimização** é o nome dado à coleção de técnicas que os compiladores podem usar para diminuir o tamanho e/ou aumentar a velocidade do código que produzem. Se pouca ou nenhuma otimização é feita, a compilação pode ser feita muito mais rapidamente do que se um esforço significativo for feito para produzir código otimizado. A escolha entre as duas alternativas é influenciada pelo ambiente no qual o compilador será usado. Em um laboratório para estudantes que estão iniciando a programação – os quais geralmente compilam seus programas

diversas vezes durante o desenvolvimento, mas que usam pouco tempo de execução de código (seus programas são pequenos e precisam ser executados corretamente apenas uma vez) – pouca ou nenhuma otimização deve ser feita. Em um ambiente de produção, onde os programas compilados são executados muitas vezes após o desenvolvimento, é melhor pagar o custo extra de otimizar o código.

O quinto fator é o custo do sistema de implementação da linguagem. Um dos fatores que explica a rápida aceitação de Java são os sistemas de compilação/interpretação gratuitos que estavam disponíveis logo após seu projeto ter sido disponibilizado ao público. Uma linguagem cujo sistema de implementação é caro ou pode ser executado apenas em plataformas de hardware caras terão uma chance muito menor de se tornarem amplamente usados. Por exemplo, o alto custo da primeira geração de compiladores Ada ajudou a prevenir que Ada tivesse se tornado popular em seus primeiros anos.

O sexto fator é o custo de uma confiabilidade baixa. Se um aplicativo de software falha em um sistema crítico, como uma usina nuclear ou uma máquina de raio X para uso médico, o custo pode ser muito alto. As falhas de sistemas não críticos também podem ser muito caras em termos de futuros negócios perdidos ou processos decorrentes de sistemas de software defeituosos.

A consideração final é o custo de manter programas, que inclui tanto as correções quanto as modificações para adicionar novas funcionalidades. O custo da manutenção de software depende de um número de características de linguagem, principalmente da legibilidade. Como que a manutenção é feita em geral por indivíduos que não são os autores originais do programa, uma legibilidade ruim pode tornar a tarefa extremamente desafiadora.

A importância da facilidade de manutenção de software não pode ser subestimada. Tem sido estimado que, para grandes sistemas de software com tempos de vida relativamente longos, os custos de manutenção podem ser tão grandes como o dobro ou o quádruplo dos custos de desenvolvimento (Sommerville, 2005).

De todos os fatores que contribuem para os custos de uma linguagem, três são os mais importantes: desenvolvimento de programas, manutenção e confiabilidade. Como esses fatores são funções da facilidade de escrita e da legibilidade, esses dois critérios de avaliação são, por sua vez, os mais importantes.

É claro, outros critérios podem ser usados para avaliar linguagens de programação. Um exemplo é a **portabilidade**, a facilidade com a qual os programas podem ser movidos de uma implementação para outra. A portabilidade é, na maioria das vezes, fortemente influenciada pelo grau de padronização da linguagem. Algumas, como o BASIC, não são padronizadas, fazendo os programas escritos nessas linguagens serem difíceis de mover de uma implementação para outra.

A padronização é um processo difícil e consome muito tempo. Um comitê começou a trabalhar em uma versão padrão de C++ em 1989, aprovada em 1998.

A **generalidade** (a aplicabilidade a uma ampla faixa de aplicações) e o fato de uma linguagem ser **bem definida** (em relação à completude e à precisão do documento oficial que define a linguagem) são outros dois critérios.

A maioria dos critérios, principalmente a legibilidade, a facilidade de escrita e a confiabilidade, não é precisamente definida nem exatamente mensurável. Independentemente disso, são conceitos úteis e fornecem ideias valiosas para o projeto e para a avaliação de linguagens de programação.

Uma nota final sobre critérios de avaliação: os critérios de projeto de linguagem têm diferentes pesos quando vistos de diferentes perspectivas. Implementadores de linguagens estão preocupados principalmente com a dificuldade de implementar as construções e recursos da linguagem. Os usuários estão preocupados primeiramente com a facilidade de escrita e depois com a legibilidade. Os projetistas são propensos a enfatizar a elegância e a habilidade de atrair um grande número de usuários. Essas características geralmente entram em conflito.

1.4 INFLUÊNCIAS NO PROJETO DE LINGUAGENS

Além dos fatores descritos na Seção 1.3, outros também influenciam o projeto básico das linguagens de programação. Os mais importantes são a arquitetura de computadores e as metodologias de projeto de programas.

1.4.1 Arquitetura de computadores

A arquitetura básica dos computadores tem um efeito profundo no projeto de linguagens. A maioria das linguagens populares dos últimos 50 anos tem sido projetada considerando a principal arquitetura de computadores, chamada de **arquitetura de von Neumann**, cujo nome é derivado de um de seus criadores, John von Neumann (pronuncia-se "von Noyman"). Elas são chamadas de linguagens **imperativas**. Em um computador von Neumann, tanto os dados quanto os programas são armazenados na mesma memória. A unidade central de processamento (CPU), que executa instruções, é separada da memória. Logo, instruções e dados devem ser transmitidos da memória para a CPU. Resultados de operações na CPU devem ser retornados para a memória. Praticamente todos os computadores digitais construídos desde os anos 1940 têm sido baseados nessa arquitetura. A estrutura geral de um computador von Neumann é mostrada na Figura 1.1.

Por causa da arquitetura de von Neumann, os recursos centrais das linguagens imperativas são as variáveis, que modelam as células de memória; as sentenças de atribuição, baseadas na operação de envio de dados e instruções (*piping*); e a forma iterativa de repetição nessa arquitetura. Os operandos em expressões são enviados da memória para a CPU, e o resultado da avaliação da expressão é enviado de volta à célula de memória representada pelo lado esquerdo da atribuição. A iteração é rápida em computadores von Neumann porque as instruções são armazenadas em células adjacentes de memória e repetir a execução de uma seção de código requer apenas uma simples instrução de desvio. Essa eficiência desencoraja o uso de recursão para repetição, embora a recursão seja às vezes mais natural.